IMEN574 / IDSC574 · PROGRAMMING FOR AX

3

AI-Assisted Programming

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: AI-Assisted Programming

01 **Build in the Editor**

Open, attach and request

02 **Run and Revise**

Execute and improve the report

03 **Save and Restore**

Finish a reusable workspace

01 · Build in the Editor

01 **Build in the Editor**

Open, attach and request

02 **Run and Revise**

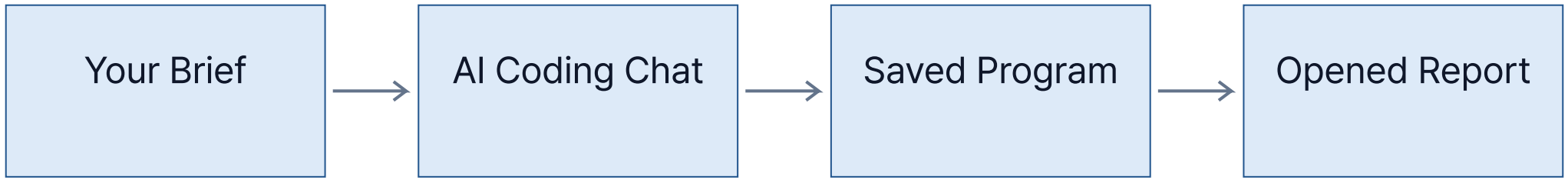
Execute and improve the report

03 **Save and Restore**

Finish a reusable workspace

Why an AI Editor

Move from a chat response to files you can run



The program turns the brief into a repeatable artifact

The Starter Folder

Three supplied files anchor the task

3_starter/

README.md: the request and acceptance rules

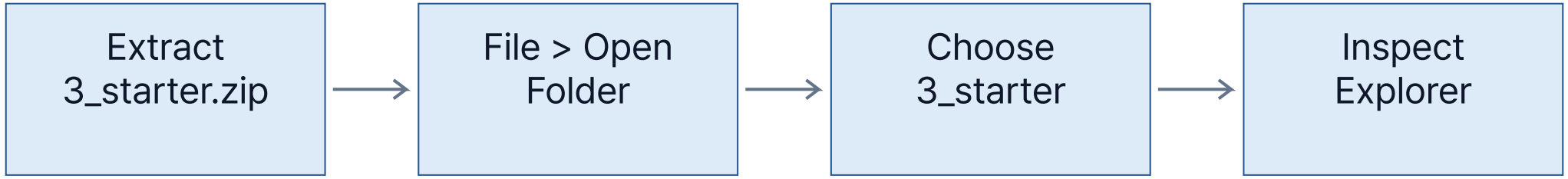
inspection.csv: eight fixed observations

checks.py: an independent result checker

The starter keeps the input and acceptance rule together

Open the Workspace

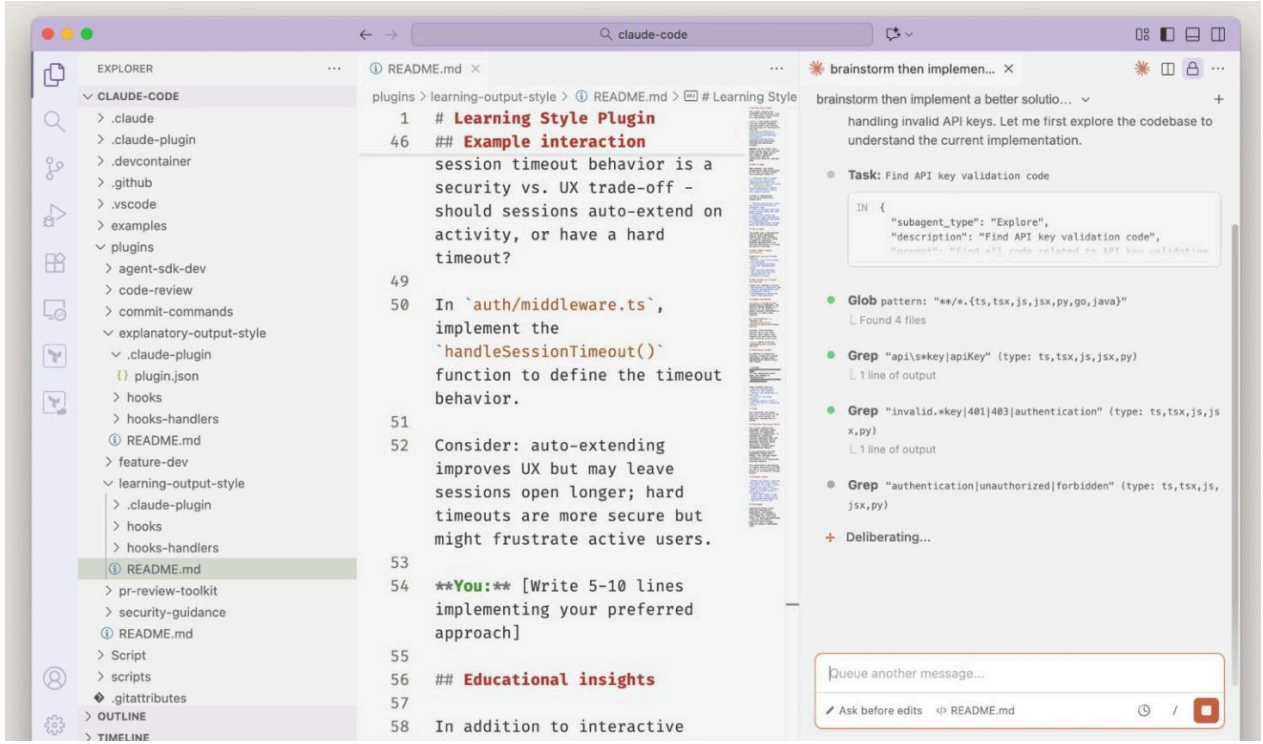
Open the extracted folder rather than one loose file



The Explorer should show all three starter files

The Editor Workspace

Files, chat and execution belong to the same working context



The editor places source files beside the AI conversation

Read the Brief First

Describe the requested result before asking for code

Input	Outputs	Boundary
inspection.csv	report.json	Keep all rows
D2 rule	inspection-report.html	Do not edit checks.py

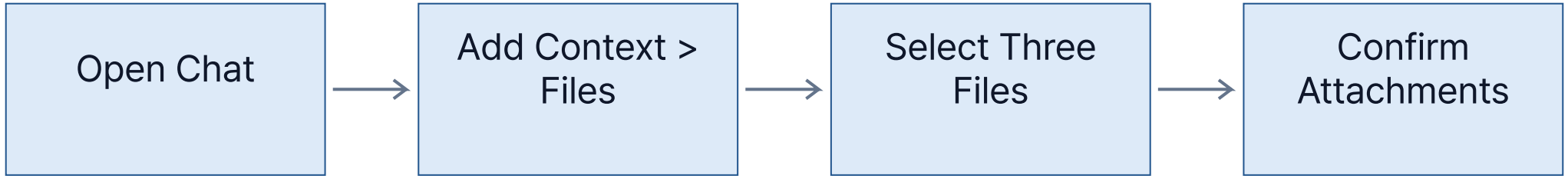
Find the Interpreter

Use the terminal to locate a working Python command

```
python --version
python3 --version
```

Attach the Task Files

Add README, CSV and checker to the coding chat

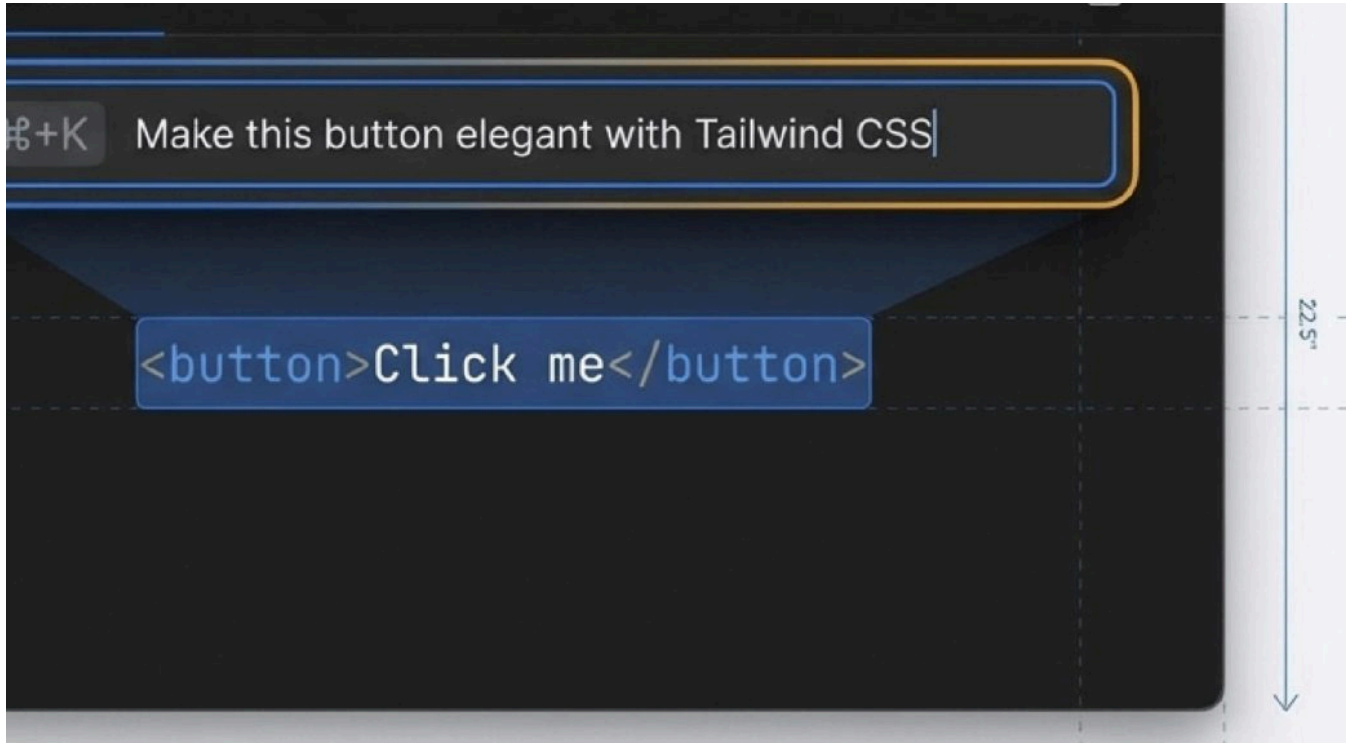


Attach the actual files before requesting implementation

https://code.visualstudio.com/docs/chat/copilot-chat-context

Ask Inside the Editor

A coding request can target the current file context



A local request can refer to the file being edited

The First Build Request

Ask for a small report generator with explicit outputs

```
Read README.md, inspection.csv and checks.py
Create app.py using only the Python standard library
Generate report.json and inspection-report.html
Keep all eight raw rows in input order
Do not modify inspection.csv or checks.py
```

A Proposed File Change

Inspect which files the assistant wants to create

Expected changes

app.py: new implementation

report.json: generated result

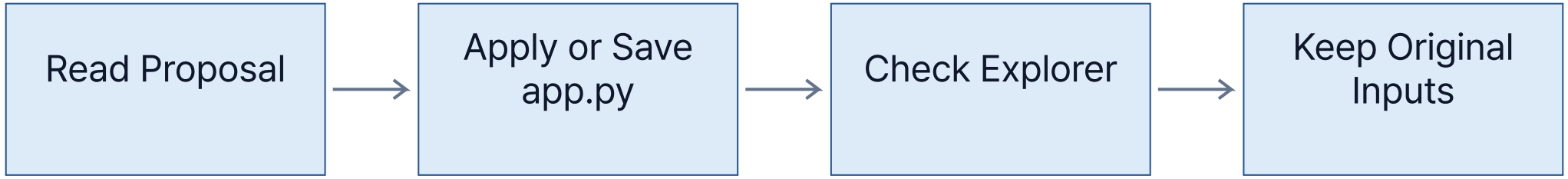
inspection-report.html: generated report

No changes to inspection.csv or checks.py

The proposed files should match the requested deliverable

Apply the Candidate

Use the editor’s change controls or save the provided file



A code block becomes useful after it is saved

02 · Run and Revise

01 **Build in the Editor**

Open, attach and request

02 **Run and Revise**

Execute and improve the report

03 **Save and Restore**

Finish a reusable workspace

Run the Program

Use one explicit command in the project terminal

```
python app.py --input inspection.csv \
  --output report.json --html inspection-report.html
```

Inspect the New Files

The Explorer should contain the requested outputs

After a successful run

app.py

report.json

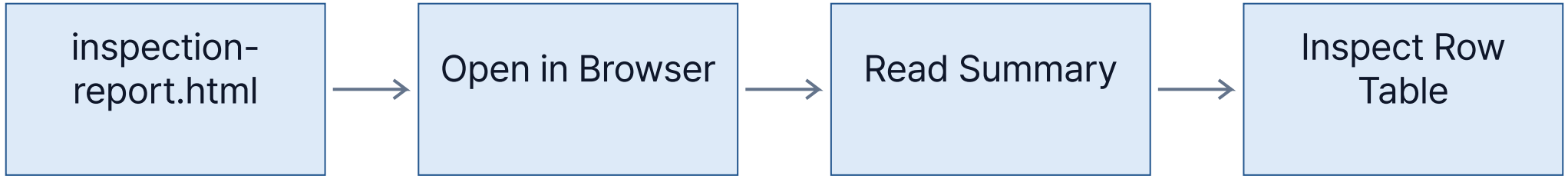
inspection-report.html

README.md, inspection.csv and checks.py unchanged

Both requested report files must exist

Open the HTML Report

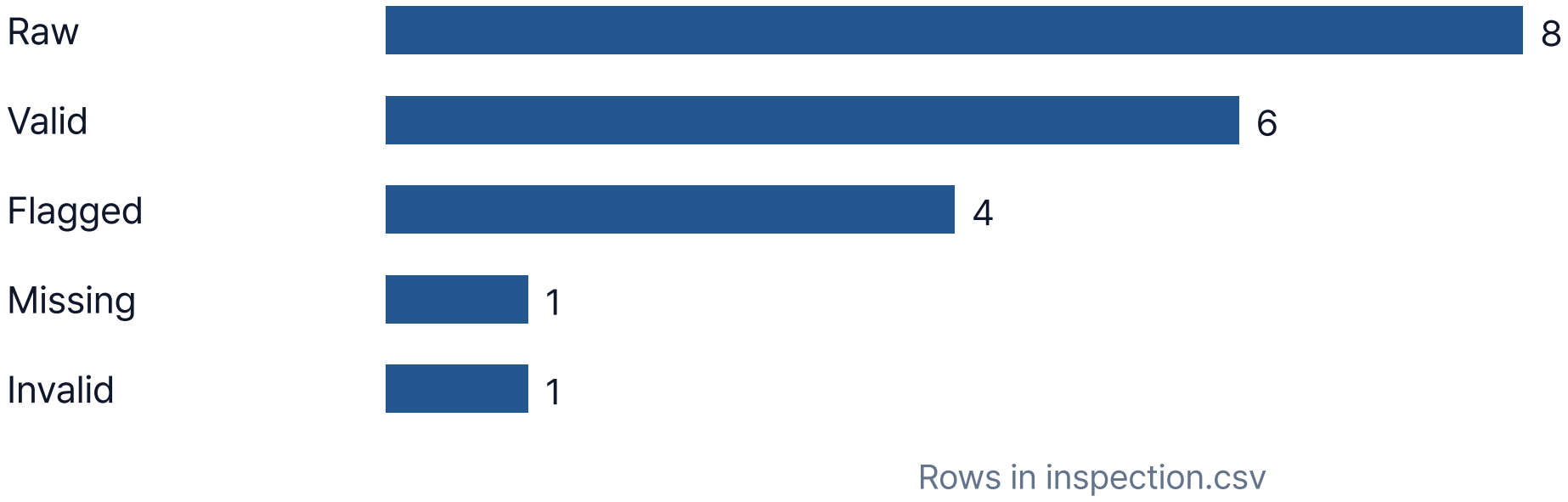
Open the generated file in a browser and read the result



The deliverable should be readable outside the editor

Read the Summary

The fixed input has a small independent reference



Eight raw rows include six valid scores

Run the Independent Checker

The checker compares the result with the source fixture

```
python checks.py
```

A Checker Failure

Use the actual error instead of guessing a cause

If a check fails

Save the exact command

Copy the actual terminal message

Keep the generated files

Attach the failing result to chat

A failure report preserves the command and observation

Demo: Repair Generated Code

Run the candidate, repair one comparison, and keep the source rows

Why are only three valid rows flagged? Repair the function and run Python

Python

```
rows = [('A01',.2),('A02',.5),('A03',.8),('A04',None),
        ('A05',1.2),('A06',.1),('A06',.6),('A08',.95)]
def flag(score):
    return score > .5
valid = [s for _,s in rows if s is not None and 0 <= s <= 1]
print('valid:', len(valid), 'flagged:', sum(flag(s) for s in valid))
print('boundary:', [(s, flag(s)) for s in [.2,.5,.8]])
```

Run Python | Reset

```
valid: 6 flagged: 3
boundary: [(0.2, False), (0.5, False), (0.8, True)]
```

Run the actual candidate against an unchanged input

Answer: A Minimal Repair

The repaired predicate includes equality and preserves six valid rows

```
def flag(score):
    return score >= .5

# valid: 6 flagged: 4
# boundary: [(0.2, False), (0.5, True), (0.8, True)]
```

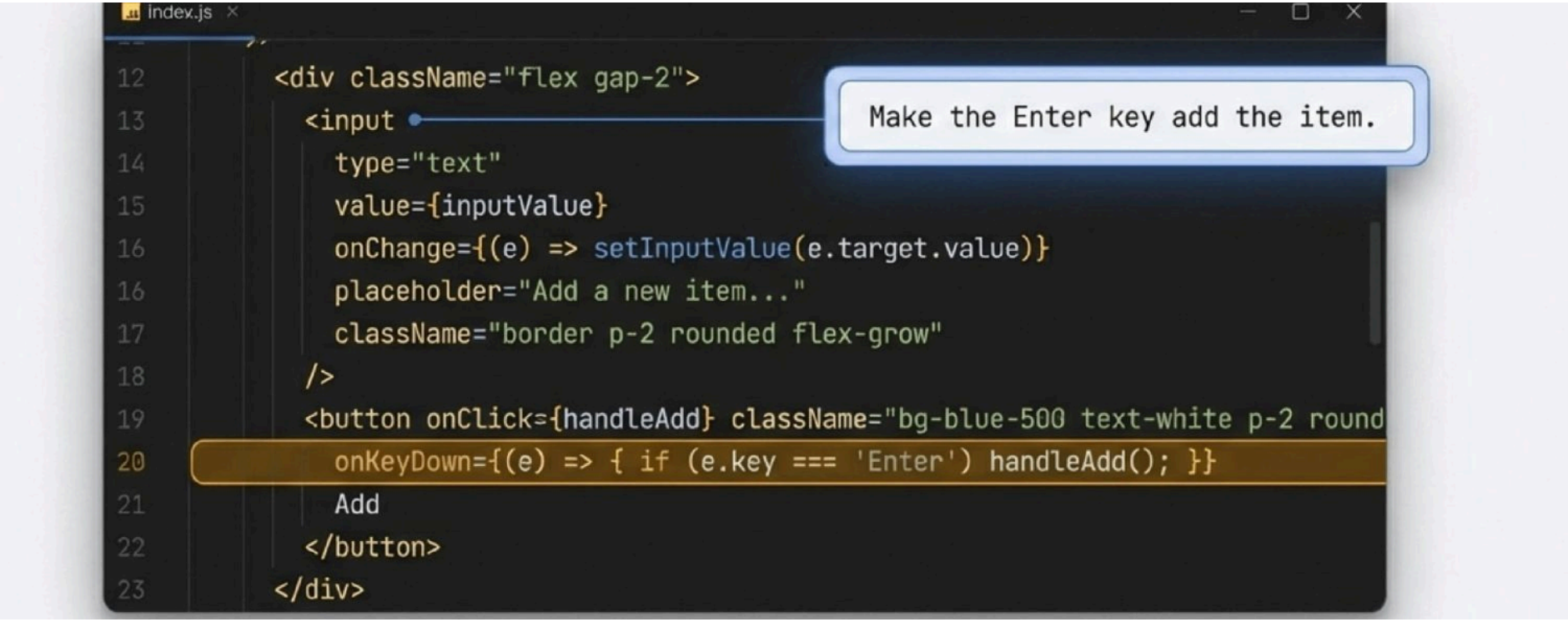
A Repair Request

Give the assistant the observed failure and preserve the contract

```
The checker fails; the actual terminal output is attached
Fix app.py so the unchanged checks.py passes
Do not change the CSV, checker or required JSON schema
Run the same generation command and checker again
```

Debugging in the Editor

The error and the relevant code can share the same conversation

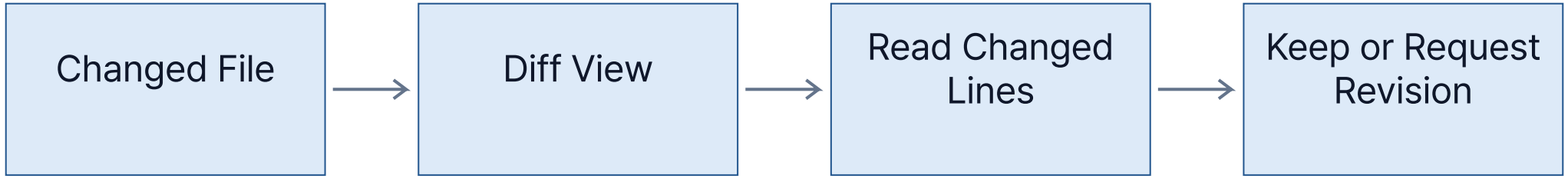


The repair request should carry the actual error context

https://code.visualstudio.com/docs/chat/copilot-chat-context

A Reviewable Diff

Compare the requested behavior with the exact edits



Diff review shows what changed without rereading everything

Input Integrity

The checker and source data remain outside the repair scope

File	Expected state
inspection.csv	Original input
checks.py	Original acceptance rule
app.py	May change
report.json and HTML	Regenerate

Demo: Preserve the Input

Improve the report order without changing the source list

The output order is correct. Why does the source-preservation check fail?

Python

```
rows = [0.2, 0.5, 0.8]
before = rows[:]
def sorted_report(values):
    values.sort(reverse=True)
    return values
report = sorted_report(rows)
print('report:', report)
print('source preserved:', rows == before)
```

Run Python | Reset

```
report: [0.8, 0.5, 0.2]
source preserved: False
```

A correct-looking output can still violate the contract

Answer: Copy Before Ordering

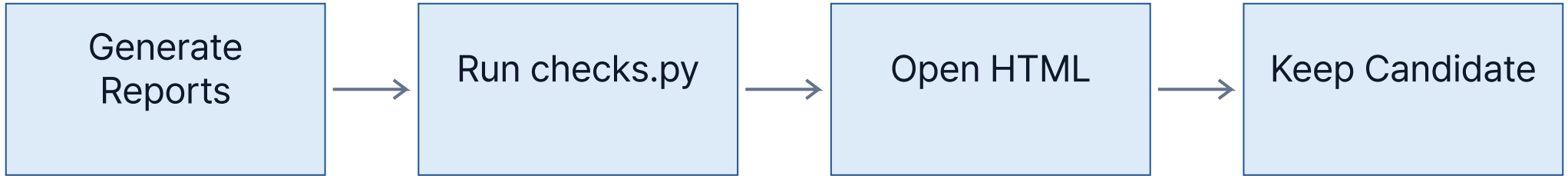
A new sorted list leaves the original input intact

```
def sorted_report(values):
    return sorted(values, reverse=True)

# report: [0.8, 0.5, 0.2]
# source preserved: True
```

A Passing Report

Accept only the result you actually opened and checked



The artifact is accepted after execution and inspection

03 · Save and Restore

01 Build in the Editor

Open, attach and request

02 Run and Revise

Execute and improve the report

03 **Save and Restore**

Finish a reusable workspace

Create a Checkpoint

Save the working state before asking for an extension

```
git init
git add README.md inspection.csv checks.py app.py
git commit -m "Working inspection report"
```

A Small Extension

Change one visible feature while preserving the data contract

```
Add the report title "Inspection Review" to the HTML
Keep report.json unchanged
Do not change row order or the checker
Generate the files and rerun checks.py
```

A Second Diff

The extension should have a smaller review surface

Before

Working report

Existing data contract

Baseline checkpoint

After

New report title

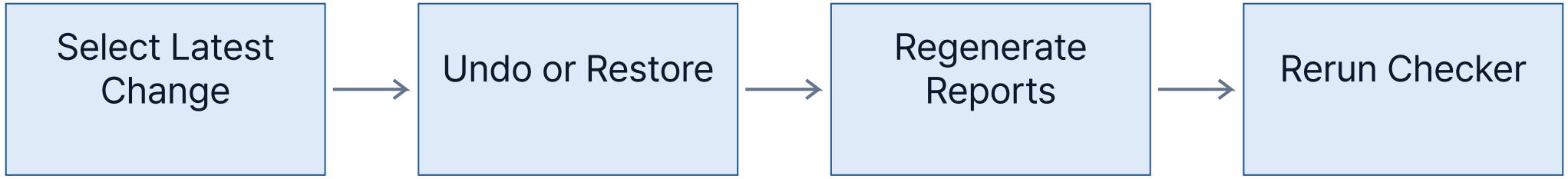
Same JSON and rows

One focused change

The extension changes presentation without changing the data result

Restore the Working Version

Undo the latest edit at the scope you intended



Restoring code requires regenerating its output

https://code.visualstudio.com/docs/agents/run/review-code-edits

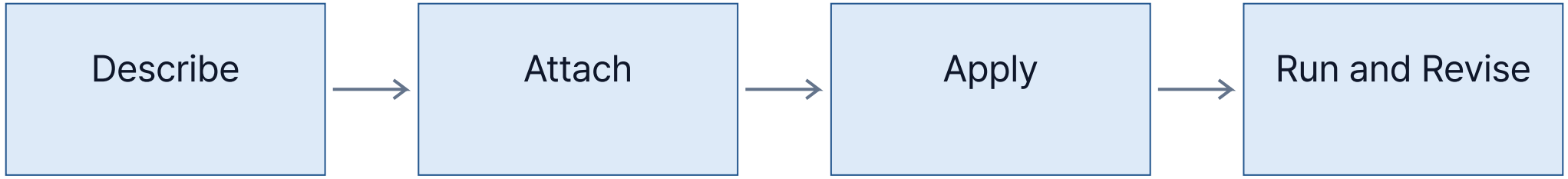
A Git Recovery Option

Restore only the exercise file from the known checkpoint

```
git diff -- app.py
git restore app.py
python app.py --input inspection.csv \
  --output report.json --html inspection-report.html
python checks.py
```

What You Already Used

The workflow precedes detailed Python syntax



The useful program exists before a syntax survey

A Reusable Project Brief

Keep durable instructions with the project files

README.md

Input and output filenames

How to run the generator

The unchanged acceptance command

The scope of permitted changes

Takeaway: AI-Assisted Programming

01 Open the whole task folder and attach the relevant files

02 Use execution feedback to request a focused repair

03 Keep a working checkpoint and a reusable artifact